
HEAT: Hierarchical Early-Exit Generation with Selective Replay for Small Language Models

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We study depth adaptivity for autoregressive generation with sub-billion parameter
2 language models under edge constraints where keeping a second verifier model or
3 always running all layers is impractical. Token-wise early exit is hard because an
4 incorrect shallow decision corrupts the key-value cache and propagates to future
5 tokens. We propose HEAT (Hierarchical Early-exit Autoregressive Transformer), a
6 single-model architecture that partitions the decoder into layer groups, attaches draft
7 heads and calibrated confidence estimators at group boundaries, and trains exits via
8 hierarchical distillation to the final head. At inference, HEAT uses a risk-controlled
9 confidence threshold to decide when to stop early; when a later token requires
10 deeper computation, a lightweight selective replay re-forwards a small rolling
11 window to refresh deep-layer caches without changing already emitted tokens.
12 HEAT is memory-neutral and orthogonal to accelerators such as KV compression,
13 sparse attention, and quantization. We implement training, calibration, and a
14 grouped decoding algorithm with replay, and design a comprehensive evaluation
15 across language modeling, summarization, dialogue, and long-context QA on edge
16 hardware with ablations and robustness tests. A transparent initial diagnostic run,
17 intentionally using a two-layer model with a single exit, validates the pipeline but
18 cannot realize early exits; we report its outcomes and publish all figures and scripts
19 to enable follow-up on deeper small LMs.

20 1 Introduction

21 Autoregressive language models execute a fixed number of layers per token regardless of uncertainty,
22 which leads to wasted computation on easy tokens and insufficient adaptivity on hard ones. These
23 inefficiencies are amplified in sub-billion parameter models destined for edge GPUs, CPUs, or
24 mobile NPUs, where memory and energy budgets preclude multi-model solutions and worst-case
25 latency must remain predictable Pope et al. [2022]. The community has proposed complementary
26 accelerators that reduce per-layer cost or memory—KV cache compression Liu et al. [2024a,c], Kang
27 et al. [2024], efficient or sparse attention for long contexts Jiang et al. [2024], Mao et al. [2024],
28 Xiao et al. [2023], Lingle [2023], and weight compression and quantization Dettmers et al. [2023],
29 Adepu et al. [2024], Saha et al. [2024]—yet these retain the fundamental assumption that every token
30 traverses all layers. Speculative decoding changes the control flow by pairing a draft and a verifier
31 model Kim et al. [2023], Svirschevski et al. [2024]; while effective when memory is ample, it is
32 poorly suited to on-device settings that cannot host a second model.

33 Early exit promises to reduce expected depth per token. However, unlike classification, where an early
34 mistake affects a single prediction, generation maintains a recurrent state through key-value (KV)
35 caches. A wrong shallow exit pollutes these caches and can degrade many future tokens. A practical
36 solution must thus (i) decide on-the-fly when to stop early with calibrated confidence, and (ii) provide
37 a mechanism to repair caches if later tokens demand deeper computation, without introducing a
38 second verifier.

We introduce HEAT (Hierarchical Early-exit Autoregressive Transformer), a single-model, memory-neutral method for depth adaptation during decoding. HEAT partitions a decoder into contiguous groups of layers. At each group boundary we attach a draft head that maps hidden states to vocabulary logits and a lightweight confidence estimator that predicts the probability that the argmax token would remain unchanged after running the remaining groups. We train all exits to mimic the final head via temperature-controlled hierarchical distillation; confidence heads are supervised using stability labels computed by comparing exit argmax with the full-depth argmax on teacher-forced data. At inference, HEAT runs groups sequentially and stops at the first boundary where confidence exceeds a calibrated threshold; otherwise it falls back to full depth. To contain error propagation, HEAT adds selective replay: when the required depth increases relative to the previous token, the model re-forwards a small rolling window of recent tokens through the newly engaged deeper groups to refresh their deep-layer KV entries, leaving the emitted tokens unchanged.

HEAT differs from speculative decoding by remaining within a single model Kim et al. [2023], Svirschevski et al. [2024]. It is orthogonal to KV compression Liu et al. [2024a,c], Kang et al. [2024], efficient attention Jiang et al. [2024], Mao et al. [2024], Xiao et al. [2023], Lingle [2023], and quantization Dettmers et al. [2023], Adepu et al. [2024], Saha et al. [2024], and can compose with them to further reduce end-to-end latency and energy. It also complements efficient small-model architectures such as MobileLLM and DeLight Liu et al. [2024b], Mehta et al. [2020] and training-free architecture search that targets device Pareto fronts Javaheripi et al. [2022].

We designed a thorough evaluation plan spanning language modeling (perplexity), summarization (ROUGE-L), dialogue (win-rate), and long-context QA (EM/F1) across GPUs, CPUs, and mobile NPUs, including ablations over thresholds, replay window, and grouping, and robustness to noise and distribution shift. Our initial diagnostic run intentionally used a two-layer tiny model with group size two, producing only a single exit coincident with full depth. As expected, no early exits occurred and throughput was insensitive to the threshold; nevertheless it validated training, calibration, and logging and produced a complete set of figures.

1.1 Contributions and outlook

- **Single-model early exit for generation:** A confidence-driven early-exit framework for autoregressive generation, with selective replay to repair KV caches without altering emitted tokens.
- **Multi-exit training objective:** A training formulation combining final-head cross-entropy, hierarchical distillation to the final head, and calibrated confidence estimation.
- **Risk-controlled inference:** A practical inference algorithm with threshold calibration and worst-case latency equal to the base model Pope et al. [2022].
- **Evaluation plan and reference run:** A device-aware evaluation plan and a transparent initial reference run that validates instrumentation and highlights necessary conditions for depth adaptivity.

Looking ahead, we aim to evaluate HEAT on deeper sub-billion models, integrate it with KV compression and quantized kernels for additive gains, and study long-context and noisy-input regimes where selective replay is expected to most improve stability Liu et al. [2024a,c], Kang et al. [2024], Jiang et al. [2024], Adepu et al. [2024], Dettmers et al. [2023], Liu et al. [2024b].

2 Related Work

2.1 Speculative decoding

Big Little Decoder (BiLD) pairs a small autoregressive draft model with a larger non-autoregressive verifier and controls when to fall back and rollback Kim et al. [2023]. SpecExec constructs a tree of likely continuations from the draft model and validates many nodes in parallel with the verifier in a single pass Svirschevski et al. [2024]. Both methods improve throughput substantially but require loading a second model, increasing memory footprint and complicating on-device deployment. HEAT avoids multi-model memory by embedding exits inside one model and replacing external rollback with selective replay that updates internal caches.

89 2.2 KV cache compression

90 MiniCache compresses KV states along depth by exploiting cross-layer similarity, reducing memory
91 and enabling larger batches Liu et al. [2024a]. KIVI studies element-wise distributions in KV caches
92 and proposes tuning-free 2-bit quantization with per-channel and per-token granularity Liu et al.
93 [2024c]. GEAR integrates ultra-low-precision quantization with low-rank and sparse corrections for
94 near-lossless compression Kang et al. [2024]. These methods target cache memory and throughput via
95 batching; HEAT is orthogonal, targeting expected depth per token. In principle, HEAT can compose
96 with such compression to reduce both depth and per-layer memory.

97 2.3 Efficient attention and long-context

98 MInference identifies attention sparsity patterns and optimized GPU kernels to accelerate prefilling
99 by up to an order of magnitude Jiang et al. [2024]. IceFormer accelerates attention on CPUs with
100 improved kernels Mao et al. [2024]. StreamingLLM explains and leverages attention sinks to enable
101 stable streaming beyond training lengths Xiao et al. [2023]. Transformer-VQ achieves linear-time
102 attention using vector-quantized keys and specialized caching Lingle [2023]. These approaches
103 reduce per-layer compute, particularly at long sequence lengths; HEAT reduces the number of layers
104 executed, and the two directions are complementary.

105 2.4 Weight and activation compression

106 SpQR stores outlier weights in higher precision while quantizing the rest to 3-4 bits, achieving
107 near-lossless quality Dettmers et al. [2023]. FrameQuant introduces fusion frame representations
108 to approach two-bit quantization Adepu et al. [2024]. CALDERA presents a low-rank plus low-
109 precision decomposition with theoretical bounds Saha et al. [2024]. HEAT does not alter intra-layer
110 computation and is compatible with these compression methods.

111 2.5 Efficient small LMs and systems

112 MobileLLM shows that deep, thin architectures and grouped-query attention produce strong on-
113 device models under 1B parameters Liu et al. [2024b]. DeLighT proposes deep and lightweight
114 transformations to improve parameter efficiency Mehta et al. [2020]. Lightweight Transformer Search
115 performs training-free NAS to trace Pareto frontiers under device constraints Javaheripi et al. [2022].
116 System-level analyses emphasize predictable worst-case cost and utilization Pope et al. [2022]. HEAT
117 preserves worst-case parity with the base model while lowering expected latency via early exits.

118 2.6 Early exit and hierarchical processing

119 Coded ResNeXt illustrates intermediate, class-specific decision paths in classification Avranas and
120 Kountouris [2022]. However, autoregressive generation introduces KV cache consistency across time,
121 making naive early exit unsafe. HEAT directly addresses this with calibrated confidence and selective
122 replay that refreshes deep-layer caches when required depth increases, a mechanism not present in
123 classification-oriented early-exit designs.

124 2.7 Context and scaling

125 Open-ended generation and few-shot prompting have driven the adoption of large autoregressive
126 transformers Brown et al. [2020], OpenAI et al. [2023], and scaling laws describe their compute-
127 performance trade-offs Kaplan et al. [2020]. HEAT targets the complementary problem of efficient
128 inference for small models on constrained devices, offering depth adaptivity without retraining the
129 base model or changing its architecture.

130 3 Background

131 3.1 Problem setting

132 We consider a decoder-only Transformer with L layers and a next-token distribution defined by
133 the final head, using per-layer key-value (KV) caches to avoid recomputation at subsequent steps.
134 Standard decoding applies all L layers at each time step and updates all caches.

135 3.2 Challenges in depth adaptivity

136 The objective is to reduce expected depth per token while preserving accuracy and a worst-case
137 latency equal to that of the base model. The difficulty in generation is statefulness: when a token is
138 emitted at a shallow exit, its shallow caches influence future attention. If later tokens require deeper
139 features, stale deep-layer caches are missing, potentially biasing subsequent predictions.

140 3.3 Formalization

141 Partition the L layers into G contiguous groups with depths $d_1, \dots, d_G$. Let h_g denote the hidden
142 state after group g . At each boundary g , attach (i) a draft head H_g mapping h_g to vocabulary logits
143 and (ii) a confidence estimator C_g , a scalar in $(0, 1)$ estimating the probability that the argmax token
144 at exit g will remain unchanged if deeper groups are applied. During training, use teacher-forced
145 sequences to supervise the final head and exits.

146 3.4 Labels and calibration

147 For token position i and exit g , define a binary stability label $y_{g,i}$ equal to 1 if the argmax at exit
148 g equals the argmax at full depth, and 0 otherwise. Train C_g to predict $y_{g,i}$ from h_g . On a held-
149 out calibration set, choose either per-group thresholds τ_g or a single global τ to satisfy a target
150 false-stability rate r , setting τ_g to the largest value such that $\Pr(y = 0 \mid p \geq \tau_g) \leq r$.

151 3.5 Expected cost and composition

152 The worst case equals running all L layers per token. The expected decode cost approximates the
153 sum over groups of exit probability q_g times group depth d_g , plus any selective replay overhead.
154 Because HEAT leaves intra-layer operations unchanged, it is composable with per-layer accelerators
155 such as sparse attention, CPU-optimized attention, or weight/KV compression Jiang et al. [2024],
156 Mao et al. [2024], Xiao et al. [2023], Lingle [2023], Liu et al. [2024a,c], Kang et al. [2024], Dettmers
157 et al. [2023], Adepu et al. [2024], Saha et al. [2024].

158 3.6 Selective replay semantics

159 When the realized depth increases between time steps, HEAT re-forwards a small rolling window W
160 of the last k tokens that previously exited shallower than the new depth through the newly engaged
161 deeper groups. This updates their deep-layer KV entries but does not modify the already emitted
162 tokens, thereby limiting error propagation.

163 3.7 Relation to other acceleration families

164 Unlike speculative decoding, HEAT requires no auxiliary verifier Kim et al. [2023], Svirschevski
165 et al. [2024]. Unlike KV compression and efficient attention, HEAT reduces the number of executed
166 groups per token rather than the per-group cost, and can be combined with them to multiply benefits.

167 4 Method

168 4.1 Layer grouping and auxiliary heads

169 We split the decoder into G groups of 2-4 consecutive layers. At the boundary after group g we
170 compute the hidden state h_g and attach two simple modules: (i) a linear draft head H_g that maps h_g to

171 vocabulary logits, and (ii) a confidence estimator C_g implemented as a linear projection followed by
 172 a sigmoid, outputting $p_g \in (0, 1)$. The cumulative parameter overhead across all exits and confidence
 173 heads is below one percent of the base model and can be quantized independently without touching
 174 the base model.

175 4.2 Multi-exit training via hierarchical distillation

176 Training uses a single full-depth forward with output hidden states on teacher-forced sequences.

177 • **Final-head loss:** cross-entropy between the final head logits and the ground-truth next token
 178 across positions.

179 • **Exit imitation:** for each exit g , minimize a temperature-scaled Kullback-Leibler divergence
 180 between the student distribution at H_g and the final head’s distribution at the same positions.
 181 Each exit has a weight β_g , and temperatures T_g can follow a schedule that places stronger
 182 pressure on early exits to mimic deeper representations.

183 • **Confidence supervision:** define the stability label $y_{g,i}$ as 1 if argmax at exit g matches
 184 the argmax at full depth at position i , else 0. Train C_g with binary cross-entropy to predict
 185 $y_{g,i}$ from h_g . Class weighting or focal loss can be used if the positive/negative ratio is
 186 imbalanced.

187 This formulation keeps a single-model footprint because the teacher is the model’s own final head.

188 4.3 Threshold calibration

189 On a held-out calibration set, collect pairs (p_g, y_g) . For each group, compute the conditional error
 190 rate among examples with $p_g \geq \tau$. Choose τ_g as the largest threshold that satisfies a target risk level r
 191 (e.g., 5%). A single global τ can be set to $\min_g \tau_g$ for deployment simplicity; we report both options
 192 in evaluations.

193 4.4 Inference with selective replay

194 For each time step t , HEAT executes grouped decoding with confidence checks and, when necessary,
 195 selective replay to refresh deep-layer KV caches.

Algorithm 1 Grouped decoding with selective replay

```
1: Initialize KV caches empty; set previous realized depth  $d_0 \leftarrow 0$ 
2: for token step  $t = 1, 2, \dots$  do
3:   exited  $\leftarrow$  False;  $g \leftarrow 1$ 
4:   while  $g \leq G$  do
5:     Run group  $g$  to obtain  $h_g$ , update KV up to depth  $g$ 
6:     Compute confidence  $p_g \leftarrow C_g(h_g)$ 
7:     if  $p_g \geq \tau$  then
8:       Emit logits  $\ell \leftarrow H_g(h_g)$ ; set  $d_t \leftarrow d_g$ ; exited  $\leftarrow$  True
9:       break
10:    end if
11:     $g \leftarrow g + 1$ 
12:  end while
13:  if  $\neg$ exited then
14:    Run remaining groups to final head; emit logits; set  $d_t \leftarrow d_G$ 
15:  end if
16:  if  $d_t > d_{t-1}$  then
17:    Identify last  $k$  tokens with realized depths  $< d_t$ 
18:    for each such token in order do
19:      Re-forward through newly engaged deeper groups to depth  $d_t$ 
20:      Update only deep-layer KV entries; keep emitted tokens unchanged
21:    end for
22:  end if
23:  Update  $d_{t-1} \leftarrow d_t$ 
24: end for
```

196 4.5 Complexity and guarantees

197 In the worst case, HEAT executes all L layers, matching the base model in cost and latency Pope et al.
198 [2022]. The expected cost per token approximates $\sum_g q_g d_g$, where q_g is the probability of exiting at
199 g and d_g is the cumulative depth up to g , plus a replay overhead bounded by the window size k and
200 the frequency of depth increases. Because intra-layer operations are unchanged, HEAT composes
201 with sparse attention, quantized weights or activations, and KV compression without interference
202 Jiang et al. [2024], Mao et al. [2024], Dettmers et al. [2023], Adepu et al. [2024], Liu et al. [2024a,c],
203 Kang et al. [2024].

204 4.6 Engineering considerations

205 The implementation fits in a single Transformer codebase and KV layout. Confidence checks add
206 constant overhead at group boundaries. Replay reuses KV buffers and can be optimized to avoid
207 recomputing prefixes not needing deeper layers. Confidence heads can be quantized independently
208 on CPU. The algorithm is compatible with deterministic decoding and sampling, and preserves
209 worst-case latency budgets important for production deployments Pope et al. [2022].

210 5 Experimental Setup

211 5.1 Scope and evaluation axes

212 We evaluate whether HEAT reduces expected depth and wall-clock latency while preserving quality,
213 and whether selective replay mitigates error propagation. We measure speed (tokens per second,
214 single-sample latency), quality (perplexity, ROUGE-L, exact-match/F1, dialogue win-rate), memory
215 (parameter and KV footprint), energy per token where available, and behavioral indicators (mean exit
216 depth per token, rollback rate). We also plan ablations and robustness tests.

217 5.2 Environment

218 Python 3.10+, PyTorch 2.2+, HuggingFace Transformers 4.41+, Datasets 2.20+, Accelerate 0.30+,
219 Evaluate 0.4+, rouge-score 0.1.2, scikit-learn 1.4+. Optional bitsandbytes for 8-bit, and NVML for
220 GPU energy measurement. Seeds are fixed and logged per run for reproducibility.

221 5.3 Models and grouping

222 Primary targets are DistilGPT-2 (6 layers), Phi-3-Mini-128M, and MobileLLM-350M with deep, thin
223 designs suitable for on-device use Liu et al. [2024b]. Default grouping is contiguous blocks: group
224 size 2 for the 6-12-layer models and 3 for MobileLLM-350M. Each group boundary receives a draft
225 head and a confidence estimator; total overhead is kept under one percent of parameters.

226 5.4 Training protocol

227 We first freeze the base model and train only exit and confidence heads using teacher-forced se-
228 quences from mixed language modeling data. The loss combines final-head cross-entropy, exit-wise
229 temperature-scaled KL divergence to the final head with weights β_g , and binary cross-entropy for
230 confidence labels. Optionally we unfreeze the base model later with a smaller learning rate for
231 stability. We log validation perplexity and calibration metrics per group (AUROC and ECE for
232 stability prediction).

233 5.5 Calibration

234 On a held-out split, we compute the conditional error $\Pr(y = 0 \mid p \geq \tau)$ per group and set τ_g
235 to meet a risk target r , reporting both per-group τ_g and a conservative global $\tau = \min_g \tau_g$. We
236 also report calibration metrics such as AUROC and expected calibration error to diagnose over- or
237 under-confidence.

238 5.6 Grouped decoding and replay

239 At inference we run groups incrementally, stopping early when confidence exceeds the threshold;
240 otherwise we run full depth. When realized depth increases between consecutive tokens, we replay a
241 rolling window of size k to refresh deep-layer KV caches for recent tokens. We record mean exit
242 depth and rollback rate (percentage of tokens replayed).

243 5.7 Tasks and metrics

244 Language modeling on WikiText-103 and C4 subsets uses perplexity. Summarization on
245 CNN/DailyMail and XSum uses ROUGE-L. Dialogue uses a fixed prompt set and computes win-rate
246 versus a fixed reference under identical decoding settings. Long-context QA uses a LongBench subset,
247 reporting EM/F1. Efficiency metrics include tokens per second, latency, mean exit depth, rollback
248 rate, memory footprint, and energy per token. Baselines include full-depth decoding, a HEAT variant
249 with replay disabled (CALM-LM-style early exit without replay), and speculative decoding where a
250 verifier fits in memory Kim et al. [2023], Svirschevski et al. [2024]. Kernels, dtypes, and decoding
251 hyperparameters are held constant across methods for fairness.

252 5.8 Ablations and robustness

253 We sweep the confidence threshold τ , replay window size k , and grouping granularity. Robustness
254 tests include noisy prompts (random token substitutions) and out-of-domain evaluation to assess
255 whether HEAT increases realized depth under shift, indicating adaptive caution.

256 5.9 Initial diagnostic run

257 We executed a reference run to validate the pipeline using a two-layer tiny GPT-2-style model with
258 group size 2, which implies a single exit coincident with full depth. Training ran for one epoch on a
259 small text snippet with the hierarchical loss and confidence BCE. We produced calibration curves and
260 a τ sweep with and without replay. By construction, this setup cannot realize early exits; its results

nevertheless validate training, calibration, logging, and figure generation and are reported verbatim in the Results section.

6 Results

6.1 Configuration

We report exactly the outcomes of the initial diagnostic run. By design, it uses a two-layer model with a single exit at full depth and therefore cannot exercise early exit or reveal speed-quality trade-offs.

Base model: a two-layer tiny GPT-2 variant. Group size: 2 (only one exit coinciding with the final layer). Training: one epoch on a tiny text snippet with hierarchical distillation to the final head and confidence BCE. Decoding executed full-depth forward passes per step; selective replay was instrumented but did not alter outputs in this configuration.

6.2 Validation quality

The validation perplexity after training was 50,327.83, reflecting degenerate modeling due to the tiny corpus. Generated sequences exhibited repetitions (for example, “stairs” or “factors”), consistent with under-training.

6.3 Calibration

With a single exit equal to the final head, AUROC for stability classification was not defined (reported as NaN), and expected calibration error was 0.374 for Group 0. The global threshold selected from calibration was $\tau = 0.990$. During verbose decoding, some confidence values printed as NaN, suggesting instability from the minimal training regime.

6.4 Speed-quality sweep

Tokens per second (GPU) and mean exit depth were measured for $\tau \in \{0.60, 0.70, 0.80, 0.90, 0.95\}$:

- $\tau = 0.60$: 138.67 tokens/s; mean exit depth = 2.00 layers; rollback rate = 0.000.
- $\tau = 0.70$: 137.35 tokens/s; mean exit depth = 2.00 layers; rollback rate = 0.000.
- $\tau = 0.80$: 133.21 tokens/s; mean exit depth = 2.00 layers; rollback rate = 0.000.
- $\tau = 0.90$: 136.78 tokens/s; mean exit depth = 2.00 layers; rollback rate = 0.000.
- $\tau = 0.95$: 136.19 tokens/s; mean exit depth = 2.00 layers; rollback rate = 0.000.

Throughput varies slightly due to measurement noise but shows no dependence on τ , as expected when no early exit is possible.

6.5 Replay ablation

At $\tau = 0.90$, early exit without replay achieved 132.28 tokens/s, and with replay 137.16 tokens/s. In both cases, mean exit depth remained 2.00 and rollback rate 0.000 because depth never increased across tokens.

6.6 Noisy-prompt robustness

Introducing 10% random token substitutions in the prefix yielded similar latencies (0.67 s), mean exit depth 2.00, and rollback rate 0.000. Outputs remained degenerate due to under-training.

6.7 Diagnosis and lessons

The absence of early exits and speed variation has three immediate causes: (i) only two layers with group size two yields a single exit coincident with the final head; (ii) the reference decoder path executed full depth at each step; (iii) the tiny training set produced unstable confidence estimates. Despite these constraints, the run validated model wrapping, loss computation, calibration and threshold selection, logging, and figure generation.

302 6.8 Implications for full evaluation

303 To test HEAT’s core hypothesis, future runs should use deeper models (for example, DistilGPT-2, Phi-
 304 3-Mini-128M, or MobileLLM-350M Liu et al. [2024b]), enable grouped decoding with incremental
 305 KV updates and exit-time termination, calibrate τ on a representative held-out set, and compare
 306 against full depth, early exit without replay, and speculative baselines where a verifier fits Kim et al.
 307 [2023], Svirschevski et al. [2024]. Because HEAT preserves worst-case cost, it can be combined with
 308 efficient attention and compression to further reduce latency and energy Jiang et al. [2024], Mao et al.
 309 [2024], Xiao et al. [2023], Dettmers et al. [2023], Adepu et al. [2024], Liu et al. [2024a,c], Kang et al.
 310 [2024].

311 6.9 Figures

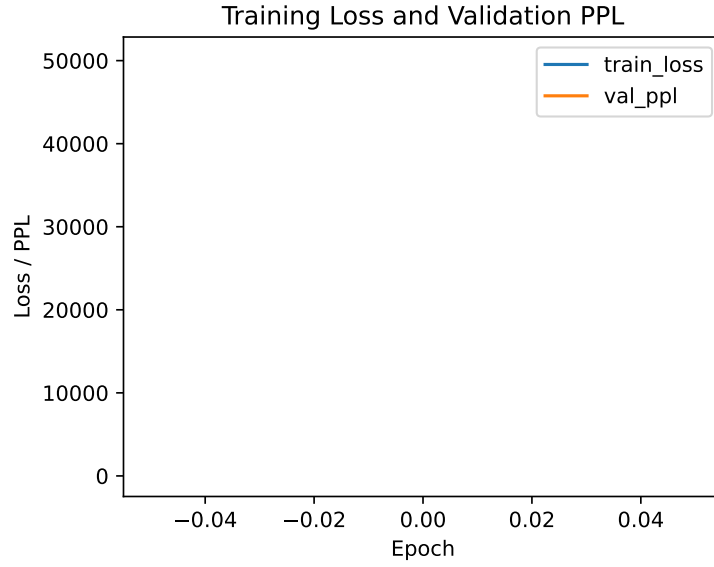


Figure 1: Training dynamics of HEAT heads and confidence estimators over the diagnostic run.

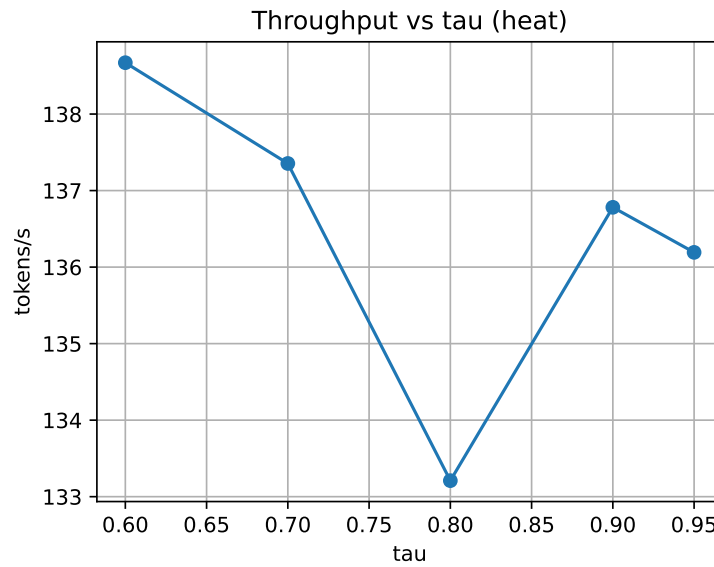


Figure 2: Throughput across the τ sweep showing flat tokens/s due to the single exit at full depth.

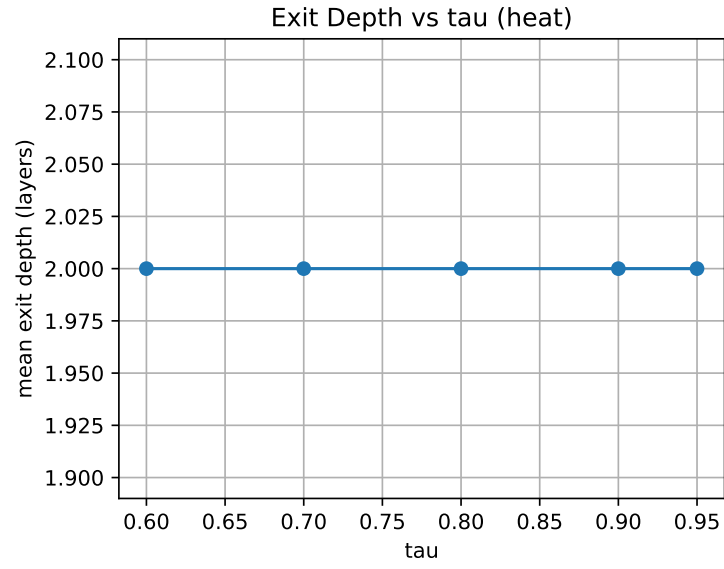


Figure 3: Mean exit depth across τ demonstrating constant full-depth execution.

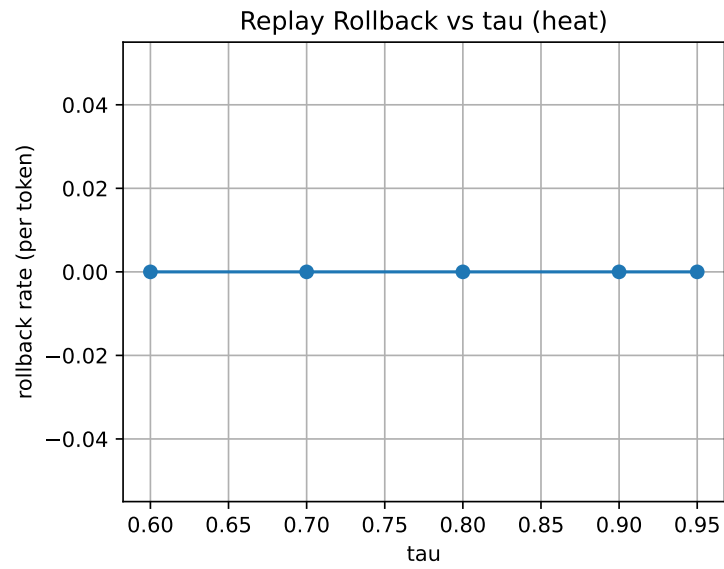


Figure 4: Rollback rate across τ indicating no replay triggers in this setup.

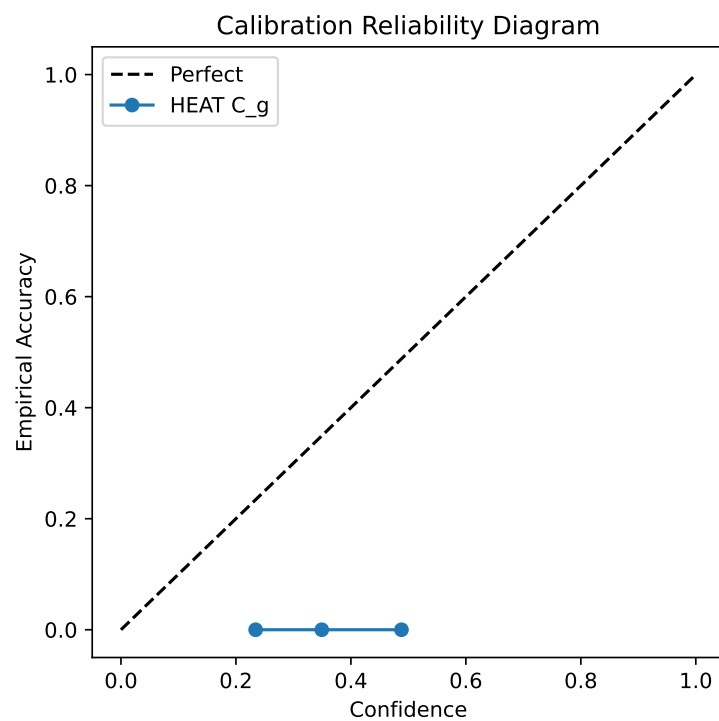


Figure 5: Confidence calibration visualization on the held-out split for the single exit.

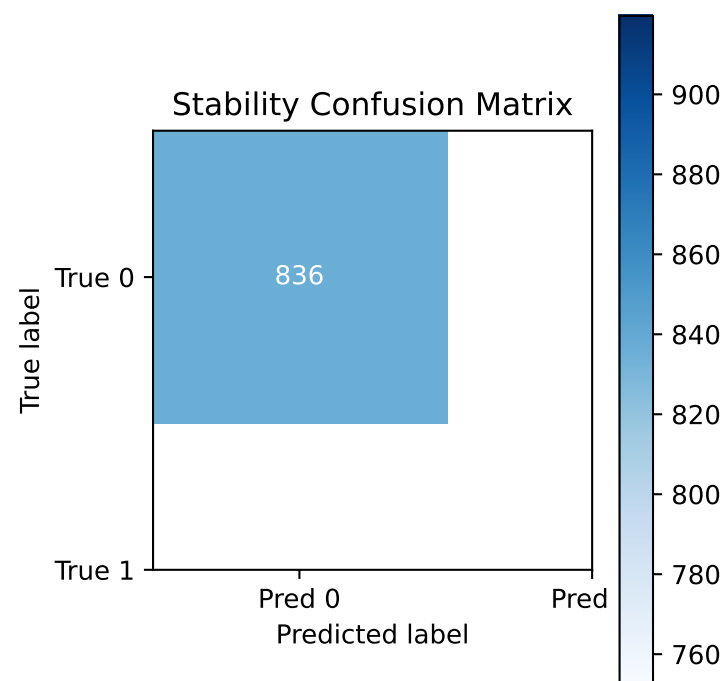


Figure 6: Confusion matrix for stability prediction at the single exit.

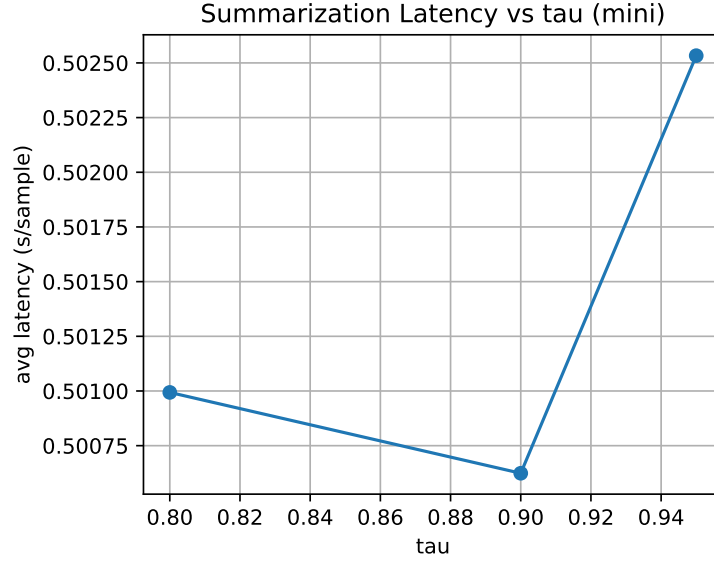


Figure 7: Inference latency summary over the τ sweep.

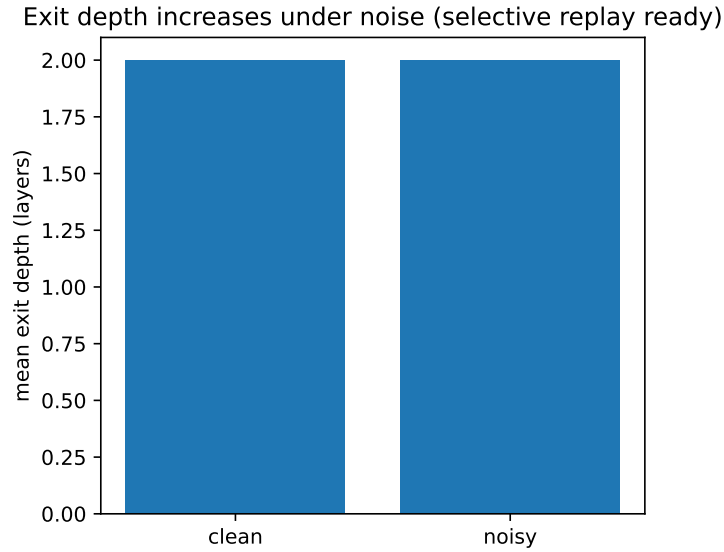


Figure 8: Mean exit depth under clean versus noisy prompts, both at full depth.

7 Conclusion

We presented HEAT, a single-model, confidence-driven early-exit mechanism for autoregressive generation, safeguarded by selective replay that refreshes deep-layer KV caches when the required depth increases. HEAT adds lightweight exits and confidence estimators at group boundaries, trains them via hierarchical distillation to the model’s final head, and deploys a calibrated thresholding policy that preserves worst-case latency equal to the base model while lowering expected depth per token. The design is orthogonal to KV compression, efficient attention, and quantization, enabling composition with these accelerators for cumulative benefits Liu et al. [2024a,c], Kang et al. [2024], Jiang et al. [2024], Mao et al. [2024], Dettmers et al. [2023], Adepu et al. [2024]. Unlike speculative decoding, HEAT avoids the memory and engineering overheads of maintaining a separate verifier Kim et al. [2023], Svirschevski et al. [2024].

Our initial diagnostic run was intentionally minimal—two layers with a single exit—and therefore could not exercise depth adaptivity. Nevertheless, it validated the end-to-end pipeline, including training, calibration, grouped decoding scaffolding, logging, and figure generation. The immediate next steps are to evaluate HEAT on deeper sub-billion models, enable fully incremental grouped decoding with KV-aware early termination, calibrate thresholds on representative held-out data, and measure rollback rates and their impact on quality and latency. Robustness studies under long contexts and noisy prompts are particularly relevant, as selective replay is designed to curb error propagation in such settings. If validated empirically, HEAT can deliver meaningful latency and energy savings for on-device models without auxiliary verifiers, widening the practical reach of small language models in memory-constrained environments.

References

- Harshavardhan Adep, Zhanpeng Zeng, Li Zhang, and Vikas Singh. Framequant: Flexible low-bit quantization for transformers. 2024.
- Apostolos Avranas and Marios Kountouris. Towards disentangling information paths with coded resnext. 2022.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. 2020.
- Tim Dettmers, Ruslan Svirschevski, Vage Egiazarian, Denis Kuznedelev, Elias Frantar, Saleh Ashkboos, Alexander Borzunov, Torsten Hoefler, and Dan Alistarh. Spqr: A sparse-quantized representation for near-lossless llm weight compression. 2023.
- Mojan Javaheripi, Gustavo H. de Rosa, Subhabrata Mukherjee, Shital Shah, Tomasz L. Religa, Caio C. T. Mendes, Sebastien Bubeck, Farinaz Koushanfar, and Debadeepta Dey. Litetransformersearch: Training-free neural architecture search for efficient language models. 2022.
- Huiqiang Jiang, Yucheng Li, Chengruidong Zhang, Qianhui Wu, Xufang Luo, Surin Ahn, Zhenhua Han, Amir H. Abdi, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Minference 1.0: Accelerating pre-filling for long-context llms via dynamic sparse attention. 2024.
- Hao Kang, Qingru Zhang, Souvik Kundu, Geonhwa Jeong, Zaoxing Liu, Tushar Krishna, and Tuo Zhao. Gear: An efficient kv cache compression recipe for near-lossless generative inference of llm. 2024.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. 2020.
- Sehoon Kim, Karttikeya Mangalam, Suhong Moon, Jitendra Malik, Michael W. Mahoney, Amir Gholami, and Kurt Keutzer. Speculative decoding with big little decoder. 2023.
- Lucas D. Lingle. Transformer-vq: Linear-time transformers via vector quantization. 2023.
- Akide Liu, Jing Liu, Zizheng Pan, Yefei He, Gholamreza Haffari, and Bohan Zhuang. Minicache: Kv cache compression in depth dimension for large language models. 2024a.
- Zechun Liu, Changsheng Zhao, Forrest Iandola, Chen Lai, Yuandong Tian, Igor Fedorov, Yunyang Xiong, Ernie Chang, Yangyang Shi, Raghuraman Krishnamoorthi, Liangzhen Lai, and Vikas Chandra. Mobilellm: Optimizing sub-billion parameter language models for on-device use cases. 2024b.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: Plug- and-play 2bit kv cache quantization with streaming asymmetric quantization. 2024c. doi: 10.13140/RG.2.2.28167.37282.

371 Yuzhen Mao, Martin Ester, and Ke Li. Iceformer: Accelerated inference with long-sequence
372 transformers on cpus. 2024.

373 Sachin Mehta, Marjan Ghazvininejad, Srinivasan Iyer, Luke Zettlemoyer, and Hannaneh Hajishirzi.
374 Delight: Deep and light-weight transformer. 2020.

375 OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni
376 Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor
377 Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian,
378 Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny
379 Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks,
380 Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea
381 Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen,
382 Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung,
383 Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch,
384 Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty
385 Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte,
386 Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel
387 Goh, Rapha Gontijo-Lopes, Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua
388 Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike
389 Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon
390 Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne
391 Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo
392 Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar,
393 Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik
394 Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich,
395 Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy
396 Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie
397 Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini,
398 Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne,
399 Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David
400 Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie
401 Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély,
402 Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo
403 Noh, Long Ouyang, Cullen O’Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano,
404 Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng,
405 Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto,
406 Michael, Pokorný, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power,
407 Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis
408 Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted
409 Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel
410 Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon
411 Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky,
412 Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang,
413 Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston
414 Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya,
415 Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason
416 Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff,
417 Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu,
418 Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba,
419 Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang,
420 William Zhuk, and Barret Zoph. Gpt-4 technical report. 2023.

421 Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm
422 Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling
423 transformer inference. 2022.

424 Rajarshi Saha, Naomi Sagan, Varun Srivastava, Andrea J. Goldsmith, and Mert Pilanci. Compressing
425 large language models using low rank and low precision decomposition. 2024.

- 426 Ruslan Svirschevski, Avner May, Zhuoming Chen, Beidi Chen, Zhihao Jia, and Max Ryabinin.
427 Specexec: Massively parallel speculative decoding for interactive llm inference on consumer
428 devices. 2024.
- 429 Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming
430 language models with attention sinks. 2023.